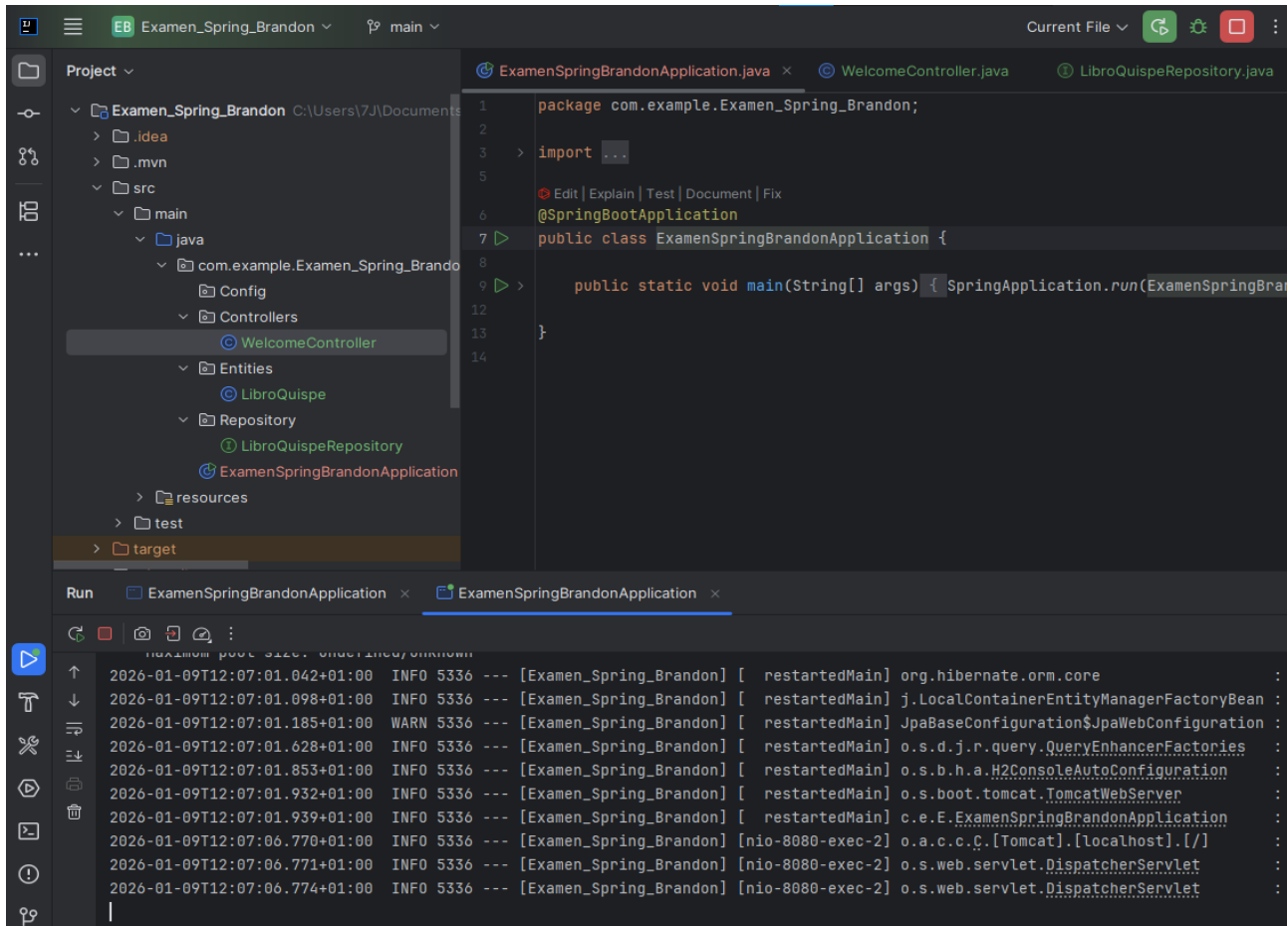


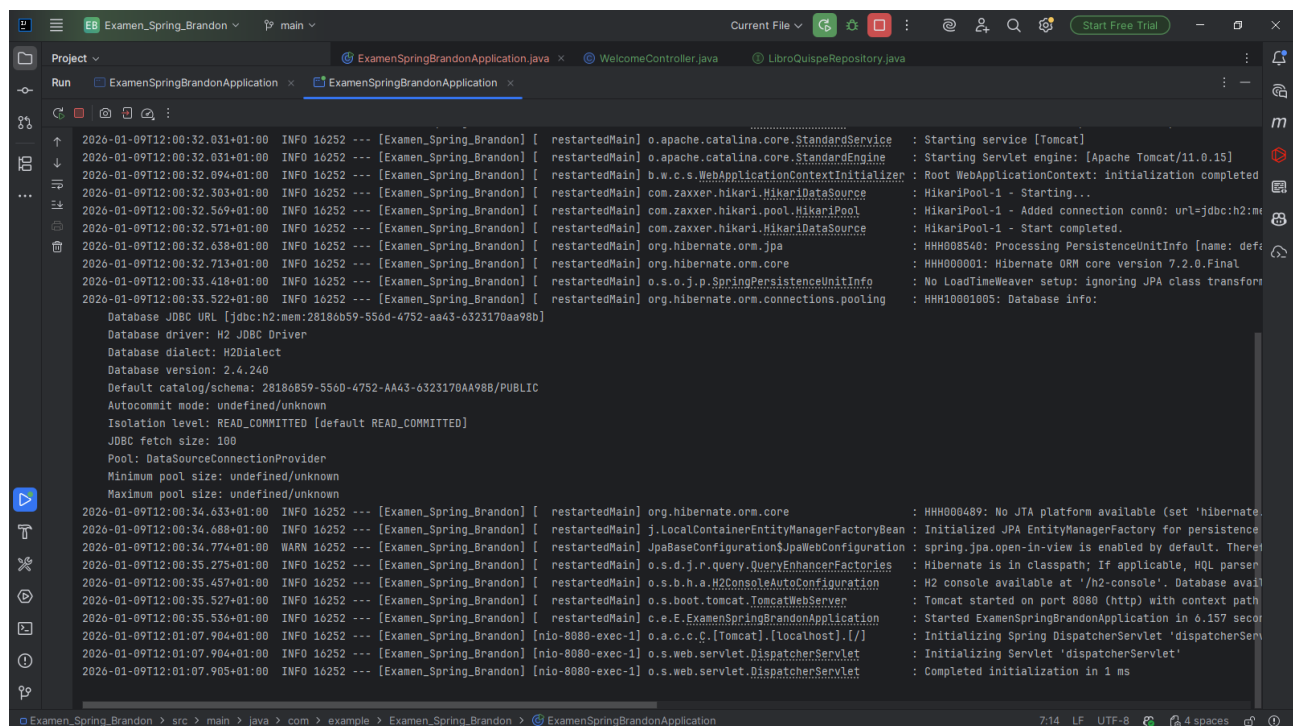
Examen Spring Brandon

CAPTURA 1

IntelliJ ejecutando la aplicación:



Consola sin errores:

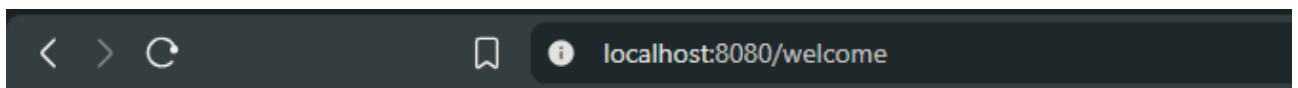


Mensaje de inicio de Spring Boot:

[illegible]

Puerto en el que se escucha Tomcat:

```
[ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 66 ms.
[ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
[ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
```



Bienvenido a la biblioteca de Brandon Quispe

Para probar el correcto funcionamiento de tomcat se ha creado un pequeño mensaje con html en el puerto /welcome

CAPTURA 2

Base de Datos:

The screenshot displays a PostgreSQL client interface. On the left, the 'Object Explorer' shows the database structure. The 'public' schema is expanded, showing a table named 'libro_quispe' with four columns: 'id' (bigint, primary key), 'anyo_publicacion' (integer), 'autor' (character varying (255)), and 'titulo' (character varying (255)).

On the right, the 'Query' tab shows a SQL query:

```
SELECT * FROM public.libro_quispe
ORDER BY id ASC
```

The 'Data Output' tab shows the result of the query, which is a single row:

	id [PK] bigint	anyo_publicacion integer	autor character varying (255)	titulo character varying (255)
1	1	1943	Antoine de Saint-Exupéry	El Principito

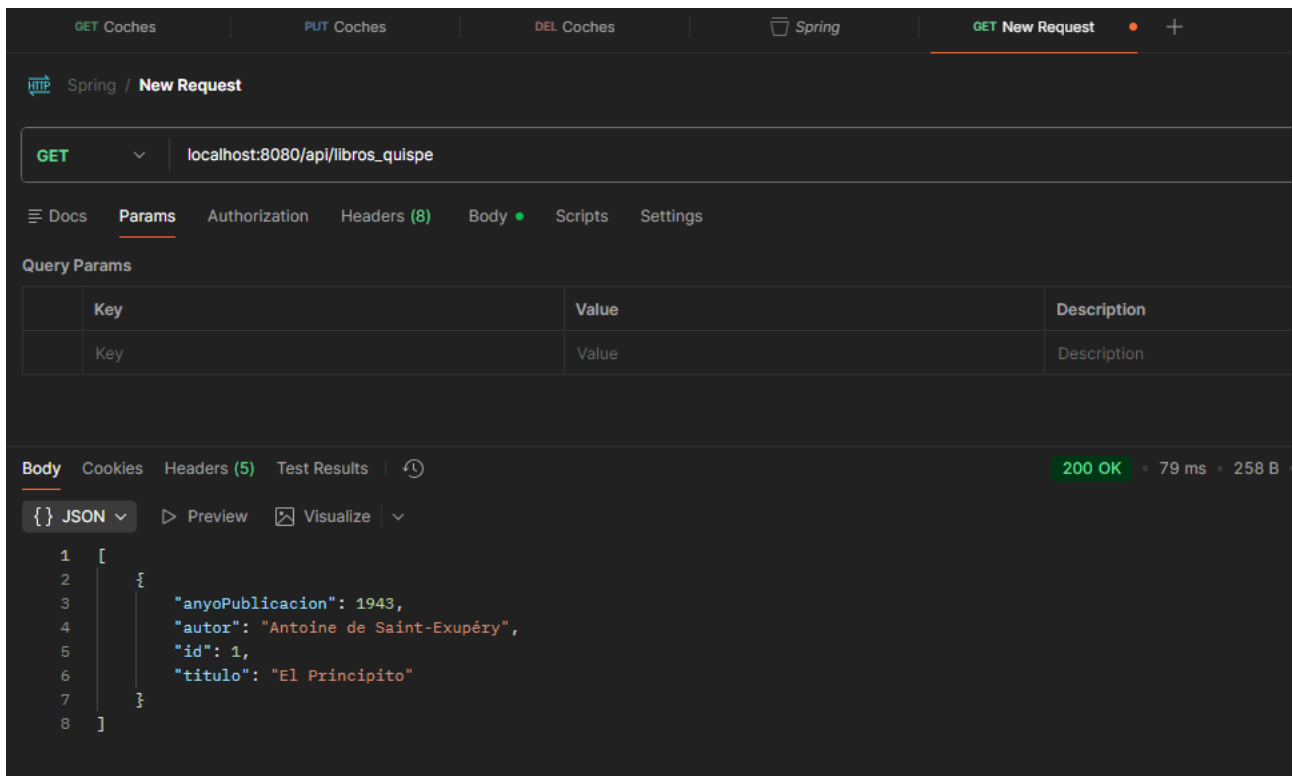
Como se observa en la imagen la base de datos ya está creada en postgresql, y contiene una tabla (libro_quispe) con 4 atributos. La base de datos se llama **biblioteca_quispe**

La tabla contiene un elemento que se generó desde spring, en un DataLoader

```
ExamenSpringBrandonApplication.java application.properties LibroQuispe.java DataLoader.java
1 package com.example.Examen_Spring_Brandon.Config;
2
3 import org.springframework.boot.CommandLineRunner;
4 import org.springframework.stereotype.Component;
5 import com.example.Examen_Spring_Brandon.Entities.LibroQuispe;
6 import com.example.Examen_Spring_Brandon.Repository.LibroQuispeRepository;
7
8 @Component
9 public class DataLoader implements CommandLineRunner {
10
11     private final LibroQuispeRepository repo;
12
13     public DataLoader(LibroQuispeRepository repo) {
14         this.repo = repo;
15     }
16
17     @Override
18     public void run(String... args) throws Exception {
19         if (repo.count() == 0) {
20             LibroQuispe libro = new LibroQuispe();
21             libro.setTitulo("El Principito");
22             libro.setAutor("Antoine de Saint-Exupéry");
23             libro.setAnyoPublicacion(1943);
24             repo.save(libro);
25             System.out.println("Libro insertado: " + libro.getTitulo());
26         }
27     }
28 }
```

CAPTURA 3

Petición GET:



The screenshot shows the Postman interface for a GET request. The URL is `localhost:8080/api/libros_quispe`. The response status is **200 OK** with a response time of 79 ms and a body size of 258 B. The response body is a JSON array containing one object.

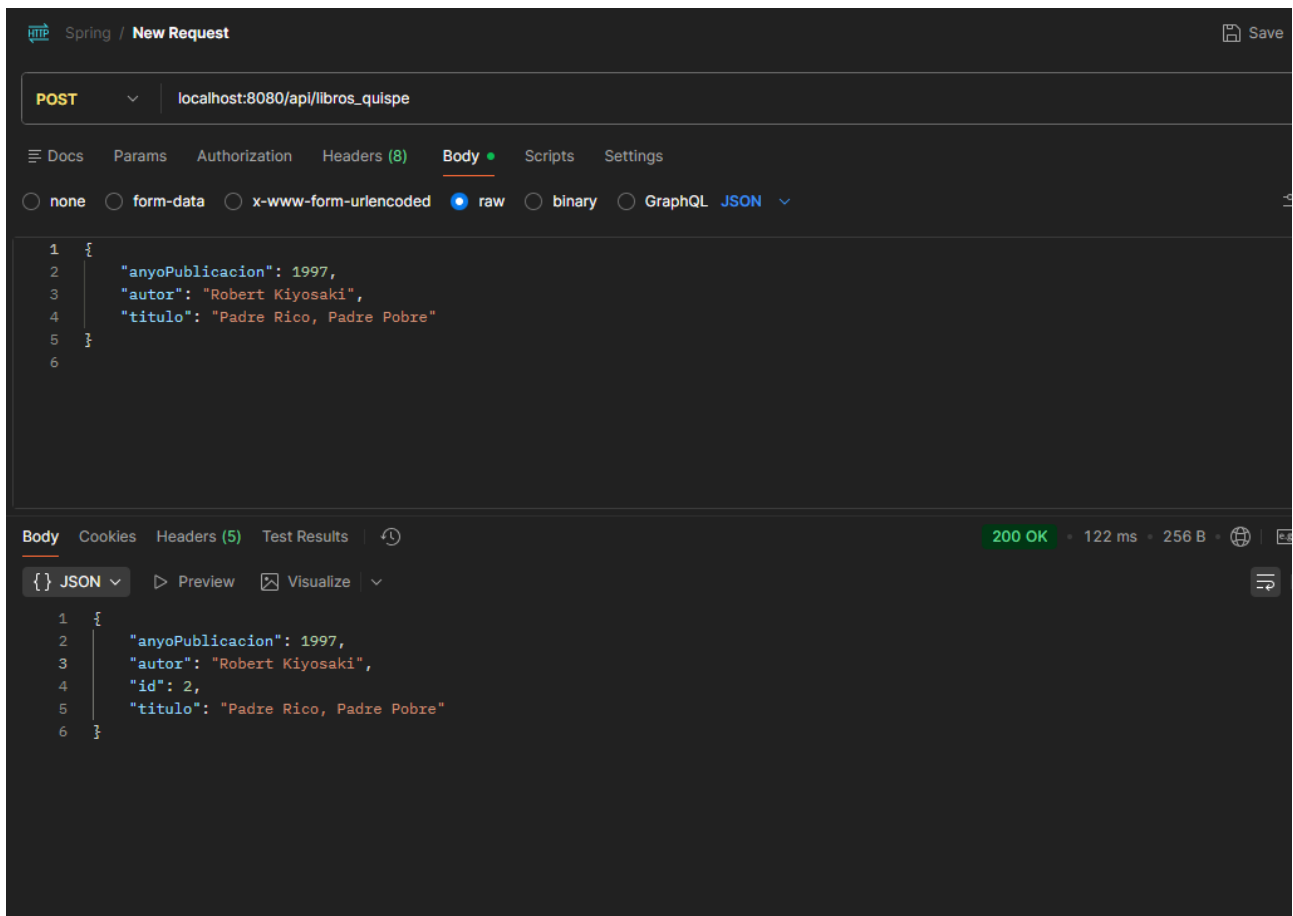
Key	Value	Description
Key	Value	Description

```
1  [
2    {
3      "anyoPublicacion": 1943,
4      "autor": "Antoine de Saint-Exupéry",
5      "id": 1,
6      "titulo": "El Principito"
7    }
8  ]
```

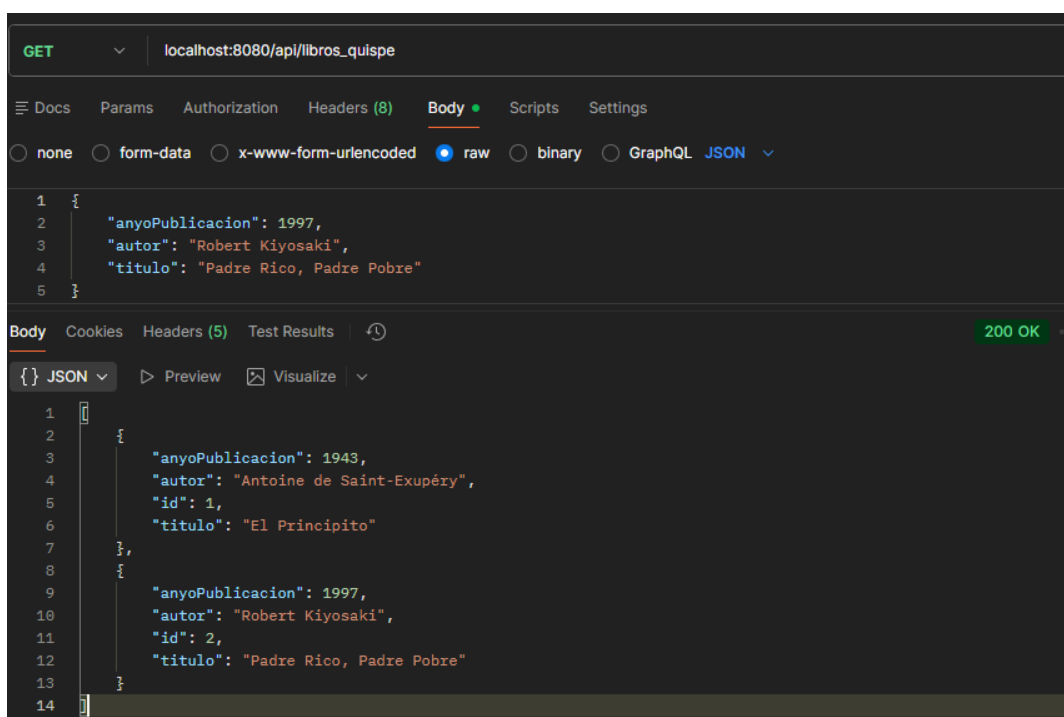
el endpoint que se usará para los diferentes métodos es “libros_quispe”, en este caso se hizo un get a través de postman, como se observa, obtuvimos un código 200, y se muestran los libros, que en este caso solo tenemos el del principito.

CAPTURA 4

Petición POST:

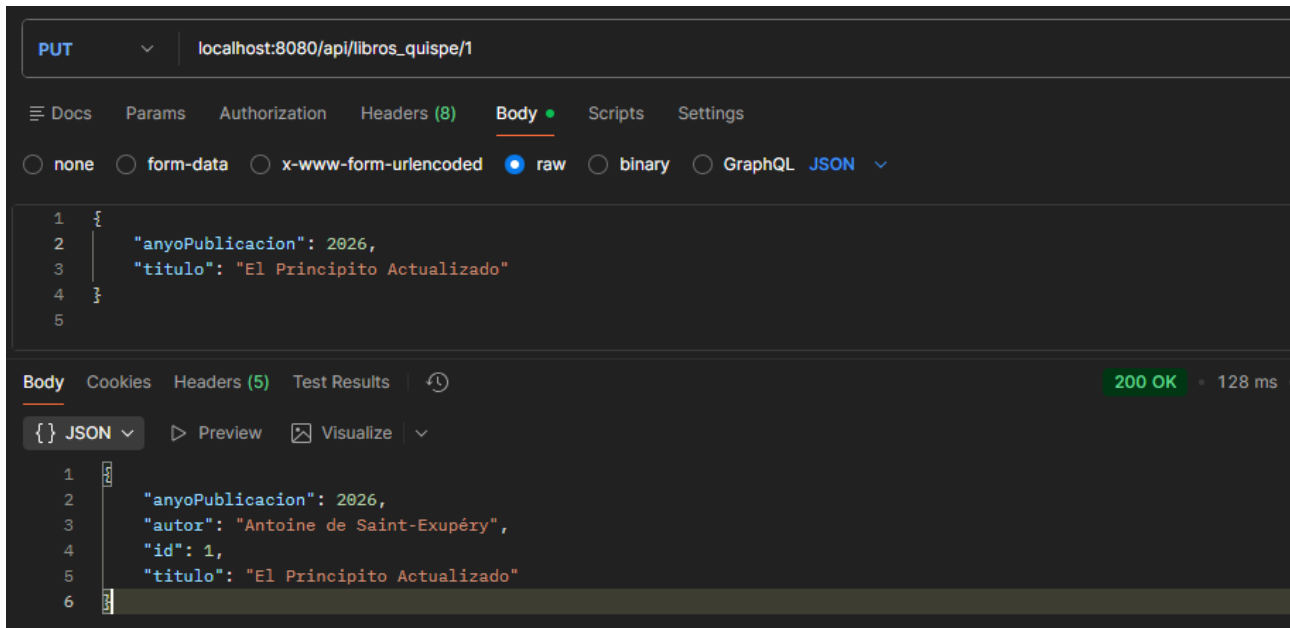


En esta ocasión se ha realizado un método post sobre el endpoint por lo que se ha añadido un nuevo libro ya que hemos recibido un código 200, hacemos una última comprobación realizando un GET rápidamente.



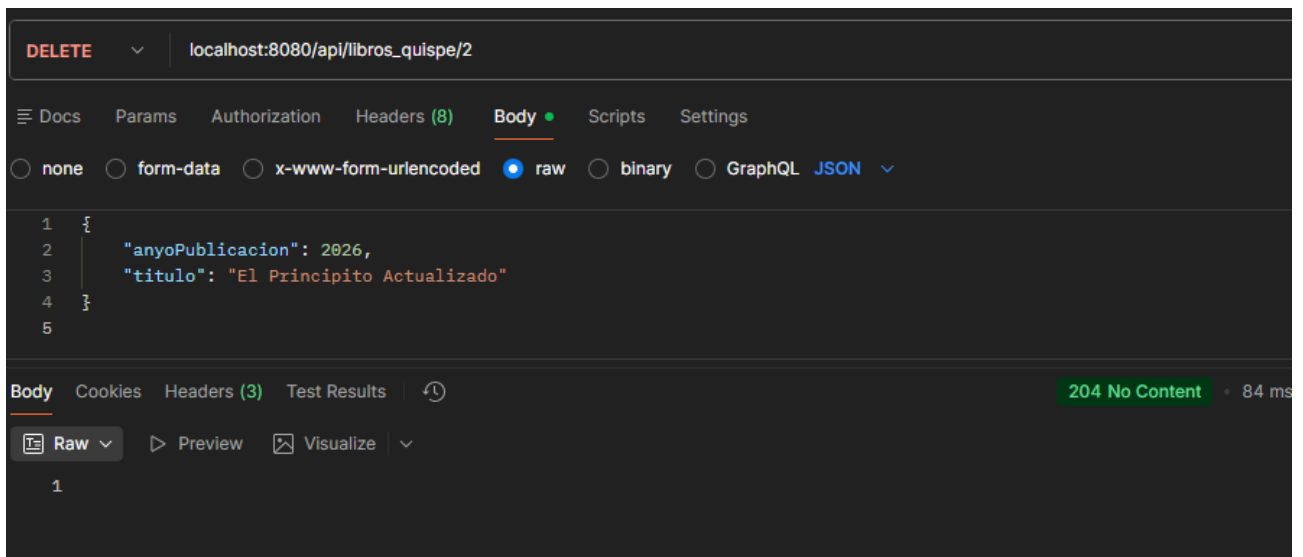
CAPTURA 5

Petición PUT:



Como vemos en la captura, específicamente en el endpoint hemos realizado una petición put sobre el libro con el id: 1 (localhost:8080/api/libros_quispe/1), por lo que hemos modificado el libro con este id, en este caso “El Principito” y le hemos modificado tanto el año de publicación como en título.

Petición DELETE:



Recibimos un 204, por lo que la petición se ha realizado correctamente, esta vez sobre el libro con el id: 2 “Padre Rico, Padre Pobre”.

Realizamos un último GET para visualizar el resultado final:

